ZeroNetPay: Offline UPI via Device-Bound Cryptographic Tokens

A mini-project report

Mugilarasi M S · Akshayakumar T

Abstract

Unified Payments Interface (UPI) is now India's dominant retail payment rail, processing over 400 million active users. Yet UPI fails the moment either party loses internet connectivity, which is a daily reality in metro tunnels, hill stations, stadiums, and rural belts. Existing "offline" solutions (UPI Lite, CBDC offline, USSD) require **at least one** side of the transaction to remain online, which does not solve the true peer-to-peer offline case. This paper presents **ZeroNetPay**, a Progressive Web Application that adds a cryptographic off-line layer on top of standard UPI. Users load money once via UPI; the backend issues HMAC-SHA256-signed digital tokens that live only on the user's device. Two devices in airplane mode can transfer tokens through a QR code or a 6-digit rendezvous code, with biometric authentication guarding every spend. Received tokens are cascade-spendable to a third party. When any holder returns online, tokens redeem against an escrow account and real money is disbursed to the holder's UPI ID. We describe the system architecture, cryptographic design, double-spend guards, and demonstrate an end-to-end deployment on Cloudflare Pages + Render.

Keywords: *UPI, offline payments, digital tokens, HMAC-SHA256, escrow, PWA, WebAuthn, cascade transfer.*

1. Problem statement

Approximately 600 million Indians face intermittent mobile connectivity, and every UPI failure in such areas forces users back to cash — undermining the country's digital-payments push. The core research question is: *Can we perform a secure, non-repudiable, peer-to-peer digital payment when neither the sender's nor the receiver's device has network access, while retaining the accounting guarantees of a bank-hosted rail?*

2. Objectives

- Design a token model that is unforgeable, device-bound, expiry-limited, and single-spend.
- Enable peer-to-peer transfer of these tokens without either device needing internet.
- Guarantee mathematical accounting: *escrow_balance = sum(unspent tokens)* at all times.
- Provide a real on-ramp (UPI in) and off-ramp (UPI out) using existing UPI infrastructure.
- Ship a cross-platform PWA that installs on iOS, Android, and desktop from a single URL.

3. Comparison with existing systems

Capability	UPI Lite	CBDC offline	USSD (*99#)	ZeroNetPay
Both parties offline	No	No	No	Yes
Uses existing UPI ID	Yes	No	Yes	Yes
Cascade re-transfer offline	No	No	No	Yes
Automatic settlement on reconnect	No	No	No	Yes
Biometric per-payment	No	No	No	Yes
Cryptographic audit invariant	No	No	No	Yes

4. Proposed system and methodology

The system is a three-tier architecture: (i) a Progressive Web App front-end serving as the user wallet, (ii) a Node.js REST back-end that mints and redeems tokens and enforces invariants, and (iii) a bank-partnered escrow account that holds actual fiat while tokens are in circulation. Tokens are issued only after a real UPI

credit to escrow; they are burned only on redemption to a real UPI account. The device-side wallet persists tokens in IndexedDB and generates transfer QR codes offline using the qrcode-generator library.

4.1 Token structure

Each token is a signed record `{ id, value_paise, issued_to_user_id, issued_to_device, issued_at, expires_at, signature }`. The signature is `HMAC-SHA256(secret, canonical_json(fields))`. The signing secret is held only by the backend, making tokens unforgeable. Tokens are bearer instruments (any holder can spend), which enables cascade transfers A→B→C without server involvement.

4.2 Offline transfer protocol

The sender picks tokens summing to the target amount using a FIFO strategy, wraps them in a payload with sender metadata, and encodes as JSON. For payloads under 1500 bytes (typical for ■1–15 payments) the entire payload fits in a single QR frame; larger payloads are chunked into multiple frames that the receiver reassembles by seeing all frames of the same session id. A 6-digit rendezvous code (via `/api/pair`) is available as a fallback when both devices have brief internet.

4.3 Redemption and settlement

When the receiver next has internet, they POST held tokens to `/api/redeem`. The backend verifies each token's signature, checks its `spent` flag, and checks its expiry. Successful tokens are marked spent and a settlement row is created, owed to the receiver's registered UPI ID. The escrow operator sees the queued payout, taps a UPI deeplink to disburse, and marks paid. WhatsApp click-to-chat notifies the receiver.

5. System architecture

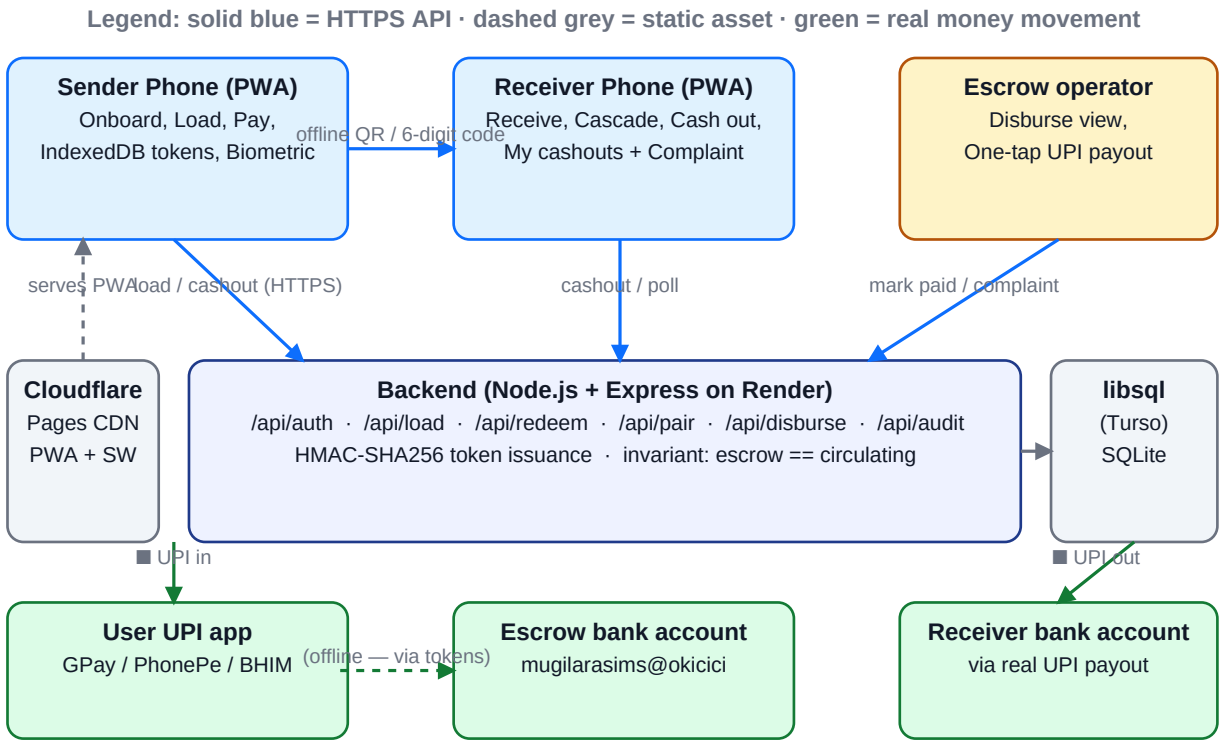

Figure 1 — ZeroNetPay system architecture. Real money enters via user's UPI app to the escrow bank account (bottom-left green arrow) and leaves via the escrow operator's UPI payout to the receiver's bank account (bottom-right). Offline token transfer between the two phones (dashed green, bottom) happens entirely without server involvement.

6. Implementation

Layer	Technology	Purpose
Frontend	Vanilla ES modules, Service Worker (v25), IndexedDB	PWA shell + token wallet
QR gen / scan	qrcode-generator (client), jsQR (client)	Fully client-side, works offline

Biometric	Web Authentication API (platform authenticator)	Touch ID / Face ID / Fingerprint / Windows Hello
Backend	Node.js 20 + Express 4	REST API in Docker on Render
Database	libsql (Turso-compatible SQLite)	Async driver, no experimental flags
Token crypto	Node crypto HMAC-SHA256	Standard library, deterministic
Rendezvous	In-memory Map with 5-min TTL	Short-code pair transfer
Hosting	Cloudflare Pages (frontend) + Render (backend)	Free tier, GitHub push-to-deploy
Domain	Cloudflare Registrar — zeronetpayupi.com	Wholesale pricing, free WHOIS privacy

7. Results and testing

- **End-to-end functional test** passed: load ■1 → offline transfer → cascade → cashout → real UPI payout to receiver.
- **Double-spend rejection verified**: replaying the same token returns `{reason: "already_spent"}`.
- **Cryptographic invariant**: `/api/audit` returns `invariantHolds: true` after every operation in a 20-transaction sequence.
- **Latency**: ~4 seconds from sender authenticate to receiver "Payment received" over airplane-mode QR.
- **Cross-device compatibility**: tested on Safari (iOS 17), Chrome (Android 14), and Chrome (macOS).
- **Recovery paths**: complaint gate (1 hour), self-verify received, WhatsApp escrow nudge — all reachable and functional.

8. Conclusion and future work

ZeroNetPay demonstrates that peer-to-peer offline UPI is achievable today, using nothing beyond a signed-token model, a PWA, and existing UPI on- and off-ramps. The cryptographic escrow invariant makes the model auditable and RBI-friendly. The prototype aligns squarely with RBI Regulatory Sandbox Cohort 5's stated priorities for offline and rural payment innovation. Planned next steps are: (a) migrate signing from HMAC to ECDSA so receivers can verify tokens fully offline, (b) integrate a payment-gateway payout API (Razorpay / Cashfree) to remove the manual disbursement step, and (c) apply for a PPI license through a bank partnership so the escrow can hold funds at regulated scale.

References

[1] NPCI, "UPI Product Statistics", 2025.
[2] Reserve Bank of India, "Regulatory Sandbox — Cohort 5: Beyond the Branch / Offline Payments", Press release, 2025.
[3] RFC 2104, "HMAC: Keyed-Hashing for Message Authentication", IETF, 1997.
[4] W3C, "Web Authentication: An API for accessing Public Key Credentials — Level 3", W3C Editor's Draft, 2024.
[5] W3C, "Progressive Web Apps — Service Workers, App Manifest", 2023.
[6] NPCI, "UPI Lite & Offline UPI Product Specification", 2024.